PROGRAMAREA CALCULATOARELOR

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 2 / 13.10.2025

PROGRAMA CURSULUI

Introducere

- Algoritmi
- Limbaje de programare.
- Introducere în C. Structura unui program C.

Fundamentele limbajului C

- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii.
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții.
- Funcții de citire/scriere.
- Etapele realizării unui program C.

Fișiere text

- Funcții specifice de manipulare.

Funcții (1)

- Declarare și definire. Apel. Metode de trasmitere a paramerilor. Pointeri la funcții.

Tablouri și pointeri

- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

Șiruri de caractere

- Funcții specifice de manipulare.

Fișiere binare

- Funcții specifice de manipulare.

Structuri de date complexe și autoreferite

- Definire și utilizare

Funcții (2)

- Funcții cu număr variabil de argumente.
- Preluarea argumentelor funcției main din linia de comandă.
- Programare generică.

CUPRINSUL CURSULUI DE AZI

1. Tipuri de date fundamentale
2. Variabile și constante
3. Expresii și operatori

TIPURI DE DATE FUNDAMENTALE

- programele manipulează date sub formă de numere, litere, cuvinte, etc.
- tipul de date specifică:
  - ✓ natura datelor care pot fi stocate în variabilele de acel tip
  - ✓ necesarul de memorie
  - ✓ operațiile permise asupra acestor variabile
- în C89, limbajul C are cinci categorii fundamentale de tipuri de date: **int**, **char**, **double**, **float**, **void**
- C99 a introdus alte 3 tipurile de date:
  - _Bool (true, false), de fapt valori întregi (0 = fals, altceva = adevărat)
  - _Complex pentru numere complexe
  - _Imaginary pentru numere imaginare

TIPURI DE DATE FUNDAMENTALE

- categorii fundamentale de tipuri de date:
  - **întreg** – **int**: variabilele de acest tip pot reține valori întregi ca 2, 0, -532
  - **caracter** – **char**: variabilele de acest tip pot reține codul ASCII al unui caracter (număr întreg) sau numere întregi mici
  - **real** (numere în virgulă mobilă) – **simplă precizie** – **float**: variabilele de acest tip pot reține numere care conțin parte fracționară: 4971.185, -0.72561, 2.000, 3.14
  - **real** (numere în virgulă mobilă) **în dublă precizie** – **double**: variabilele de acest tip pot reține valori reale în virgulă mobilă cu o precizie mai mare decât tipul float
  - **void**: indică lipsa unui tip anume

TIPURI DE DATE FUNDAMENTALE

- se pot crea noi tipuri de date prin combinarea celor de bază
- reprezentarea și spațiul ocupat în memorie de diferitele tipuri de date depind de: platformă, sistem de operare, compilator
- limitele specifice unui sistem de calcul pot fi aflate din fișierele header **limits.h** și **float.h**
  - *exemplu:* CHAR_MAX, INT_MAX, INT_MIN, FLT_MAX, DBL_MAX
- pentru determinarea numărului de octeți ocupați de un anumit tip de date se folosește operatorul **sizeof**
  - ✓ 1 octet = 1 byte = 8 biți

SPAȚIUL OCUPAT ÎN MEMORIE

```c
#include<stdio.h>

int main()
{
    printf("nr octeti (char)= %ld\n", sizeof(char));
    printf("nr octeti (int)= %ld\n", sizeof(int));
    printf("nr octeti (float)= %ld\n", sizeof(float));
    printf("nr octeti (double)= %ld\n", sizeof(double));

    return 0;
}
```

```
nr octeti (char)= 1
nr octeti (int)= 4
nr octeti (float)= 4
nr octeti (double)= 8
```

SPAȚIUL OCUPAT ÎN MEMORIE

```c
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("\nPentru tipul char:\n");
    printf("nr octeti = %d\n", sizeof(char));
    printf("valoarea maxima = %d\n", CHAR_MAX);
    printf("valoarea minima = %d\n", CHAR_MIN);

    printf("\nPentru tipul int:\n");
    printf("nr octeti = %d\n", sizeof(int));
    printf("valoarea maxima = %d\n", INT_MAX);
    printf("valoarea minima = %d\n", INT_MIN);

    return 0;
}
```

SPAȚIUL OCUPAT ÎN MEMORIE

```c
#include <stdio.h>
#include <limits.h>

int main()
{
    printf("\nPentru tipul char:\n");
    printf("nr octeti = %d\n", sizeof(char));
    printf("valoarea maxima = %d\n", CHAR_MAX);
    printf("valoarea minima = %d\n", CHAR_MIN);

    printf("\nPentru tipul int:\n");
    printf("nr octeti = %d\n", sizeof(int));
    printf("valoarea maxima =
    printf("valoarea minima =

    return 0;
}
```

```
Pentru tipul char:
nr octeti = 1
valoarea maxima = 127
valoarea minima = -128

Pentru tipul int:
nr octeti = 4
valoarea maxima = 2147483647
valoarea minima = -2147483648
```

SPAȚIUL OCUPAT ÎN MEMORIE

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main()
{
    printf("\nPentru tipul float:\n");
    printf("nr octeti = %d\n", sizeof(float));
    printf("valoarea maxima = %E\n", FLT_MAX);
    printf("valoarea maxima = %lf\n", FLT_MAX);
    printf("valoarea minima > 0 este %E\n", FLT_MIN);
    printf("precizia este de %d zecimale\n", FLT_DIG);

    printf("\nPentru tipul double:\n");
    printf("nr octeti = %d\n", sizeof(double));
    printf("valoarea maxima = %E\n", DBL_MAX);
    printf("valoarea maxima = %lf\n", DBL_MAX);
    printf("valoarea minima > 0 este %E\n", DBL_MIN);
    printf("precizia este de %d zecimale\n", DBL_DIG);

    return 0;
}
```

SPAȚIUL OCUPAT ÎN MEMORIE

```
Pentru tipul float:
nr octeti = 4
valoarea maxima = 3.402823E+38
valoarea maxima = 340282346638528859811704183484516925440.000000
valoarea minima > 0 este 1.175494E-38
precizia este de 6 zecimale

Pentru tipul double:
nr octeti = 8
valoarea maxima = 1.797693E+308
valoarea maxima = 179769313486231570814527423731704356798070567525844996598917476803157260780028538760589558632766878171540458953514382464234321326889464182768467546703537516986049910576551282076245490903893289440758685084551339423045832369032229481658085593321233482747978262041447231687381771809192998812504040261841248583680.000000
valoarea minima > 0 este 2.225074E-308
precizia este de 15 zecimale
```

SPAȚIUL OCUPAT ÎN MEMORIE

```c
#include<stdio.h>
#include<values.h>

int main( void )
{
    printf("%d %d %ld", MAXINT, MININT, MAXLONG);
    printf("\n\n%f %f", MAXFLOAT, MINFLOAT);
    printf("\n\n%f %f", MAXDOUBLE, MINDOUBLE);
    return 0;
}
```

```
2147483647 -2147483648 9223372036854775807

340282346638528859811704183484516925440.000000 0.000000

179769313486231570814527423731704356798070567525844996598
917476803157260780028538760589558632766878171540458953514
382464234321326889464182768467546703537516986049910576551
282076245490090389328944075868508455133942304583236903222
948165808559332123348274797826204144723168738177180919299
881250404026184124858368.000000 0.000000
```

BAZE DE NUMERAȚIE

- litera corespunzătoare bazei de numerație:

  B - binar (ex. 10011101**B**)

  Q - octal (ex. 23701**Q**)

  D - zecimal (ex. 5429**D**, 5429)

  H - hexazecimal (ex. FD37B**H**)

- Baza sau indice = baza în paranteză

  Număr$_{(2)}$ - binar (ex. 101101$_{(2)}$)

  Număr(2) - binar (ex. 101101(2))

  Număr(8) - octal (ex. 5572(8))

  Număr(10) - zecimal (ex. 9334(10))

  Număr(16) - hexazecimal (ex. 53FD1(16))

Zecimal	Binar
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

BAZE DE NUMERAȚIE

- litera corespunzătoare bazei de numerație:

  B - binar (ex. 10011101**B**)

  Q - octal (ex. 23701**Q**)

  D - zecimal (ex. 5429**D**, 5429)

  H - hexazecimal (ex. FD37B**H**)

- Baza sau indice = baza în paranteză

  Număr$_{(2)}$ - binar (ex. 101101$_{(2)}$)

  Număr(2) - binar (ex. 101101(2))

  Număr(8) - octal (ex. 5572(8))

  Număr(10) - zecimal (ex. 9334(10))

  Număr(16) - hexazecimal (ex. 53FD1(16))

Zecimal	Binar	Octal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	10
9	1001	11
10	1010	12
11	1011	13
12	1100	14
13	1101	15
14	1110	16
15	1111	17

BAZE DE NUMERAȚIE

- litera corespunzătoare bazei de numerație:

  B - binar (ex. 10011101**B**)

  Q - octal (ex. 23701**Q**)

  D - zecimal (ex. 5429**D**, 5429)

  H - hexazecimal (ex. FD37B**H**)

- Baza sau indice = baza în paranteză

  Număr$_{(2)}$ - binar (ex. 101101$_{(2)}$)

  Număr(2) - binar (ex. 101101(2))

  Număr(8) - octal (ex. 5572(8))

  Număr(10) - zecimal (ex. 9334(10))

  Număr(16) - hexazecimal (ex. 53FD1(16))

Zecimal	Binar	Octal	Hexazecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

BAZE DE NUMERAȚIE

- conversia unui număr din baza 10 în altă bază de numerație:
  - împărțirea succesivă a numărului respectiv la baza în care se dorește conversia: se împarte numărul la bază, iar în continuare câtul obținut se împarte la bază până când acesta devine zero.
  - Rezultatul final se obține prin scrierea resturilor fiecărei împărțiri, în ordine inversă.

Binar
$173 : 2 = 86+1$
$86 : 2 = 43+0$
$43 : 2 = 21+1$
$21 : 2 = 10+1$
$10 : 2 = 5+0$
$5 : 2 = 2+1$
$2 : 2 = 1+0$
$1 : 2 = 0+1$
$10101101_{(2)}$

$173 = 1*2^7 + 0*2^6 + 1*2^5 + 0*2^4 + 1*2^3 + 1*2^2 + + 0*2^1 + 1*2^0$

BAZE DE NUMERAȚIE

- conversia unui număr din baza 10 în altă bază de numerație:
  - împărțirea succesivă a numărului respectiv la baza în care se dorește conversia: se împarte numărul la bază, iar în continuare câtul obținut se împarte la bază până când acesta devine zero.
  - Rezultatul final se obține prin scrierea resturilor fiecărei împărțiri, în ordine inversă.

Binar	Octal
$173 : 2 = 86+1$	$173 : 8 = 21+5$
$86 : 2 = 43+0$	$21 : 8 = 2+5$
$43 : 2 = 21+1$	$2 : 8 = 0+2$
$21 : 2 = 10+1$	
$10 : 2 = 5+0$	
$5 : 2 = 2+1$	
$2 : 2 = 1+0$	
$1 : 2 = 0+1$	
$10101101_{(2)}$	$255_{(8)}$

$173 = 2*8^2 + 5*8^1 + 5*8^0$
$= 2*64 + 5*8 + 5*1$
$= 128 + 40 + 5$

BAZE DE NUMERAȚIE

- conversia unui număr din baza 10 în altă bază de numerație:
  - împărțirea succesivă a numărului respectiv la baza în care se dorește conversia: se împarte numărul la bază, iar în continuare câtul obținut se împarte la bază până când acesta devine zero.
  - Rezultatul final se obține prin scrierea resturilor fiecărei împărțiri, în ordine inversă.

Binar	Octal	Hexazecimal
$173 : 2 = 86+1$ $86 : 2 = 43+0$ $43 : 2 = 21+1$ $21 : 2 = 10+1$ $10 : 2 = 5+0$ $5 : 2 = 2+1$ $2 : 2 = 1+0$ $1 : 2 = 0+1$	$173 : 8 = 21+5$ $21 : 8 = 2+5$ $2 : 8 = 0+2$	$173 : 16 = 10+13$ $10 : 16 = 0+10$ $173 = 10*16^1 + 13*16^0$ $= 160 + 13$
$10101101_{(2)}$	$255_{(8)}$	$AD_{(16)}$

BAZA DE NUMERAȚIE 2

Transformarea unui număr natural din baza 10 în baza 2:

98 : 2 = 49 + 0
49 : 2 = 24 + 1
24 : 2 = 12 + 0
12 : 2 = 6 + 0
6 : 2 = 3 + 0
3 : 2 = 1 + 1
1 : 2 = 0 + 1



$98_{(10)} = 1100010_{(2)}$

PUTERILE LUI 2

$2^0 = 1$

$2^1 = 2$

$2^2 = 2 \times 2 = 4$

$2^3 = 2 \times 2 \times 2 = 8$

$2^4 = 2 \times 2 \times 2 \times 2 = 4 \times 4 = 16$

$2^5 = 4 \times 4 \times 2 = 16 \times 2 = 32$

$2^6 = 32 \times 2 = 64$

$2^7 = 64 \times 2 = 128$

$2^8 = 128 \times 2 = 256$

$2^9 = 256 \times 2 = 512$

$2^{10} = 512 \times 2 = 1024$

BAZA DE NUMERAȚIE 2

- Transformarea unui număr natural din baza 10 în baza 2
  - Exemplu
    - 157:2=78+ **1**
    - 78:2 =39+ **0**
    - 39:2 =19+ **1**
    - 19:2 =9 + **1**
    - 9:2 =4 + **1**
    - 4:2 =2 + **0**
    - 2:2 =1 + **0**
    - 1:2 =0 + **1**



$157_{(10)}=10011101_{(2)}$

- Transformarea unui număr natural din baza 2 în baza 10
  - Exemplu

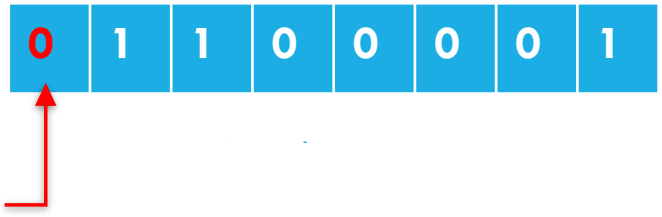
7	6	5	4	3	2	1	0
1	**0**	**0**	**1**	**1**	**1**	**0**	**1**

$10011101_{(2)}=1*2^0+0*2^1+1*2^2+1*2^3+1*2^4+0*2^5+0*2^6+1*2^7=157_{(10)}$

Reprezentarea în memorie

- **char**: ocupă 1 octet = 8 biți, valori între $-2^7=-128$ și $2^7-1=127$

char ch = ‘a’;
‘a’ are codul ASCII 97 = $2^6 + 2^5 + 2^0$



Reprezentarea întregilor cu semn: **COMPLEMENT FAȚĂ DE 2**

Reprezentarea lui +97:



1. Complementarea tuturor biților:



2. Adunare cu 1:



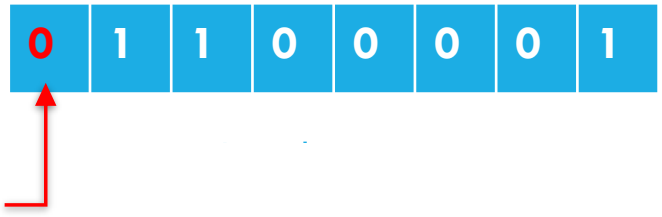
Reprezentarea lui -97:



Reprezentarea în memorie

- **char**: ocupă 1 octet = 8 biți, valori între $-2^7=-128$ și $2^7-1=127$

char ch = ‘a’;
‘a’ are codul ASCII 97 = $2^6 + 2^5 + 2^0$



Reprezentarea întregilor cu semn: **COMPLEMENT FAȚĂ DE 2**

Reprezentarea lui +97:



Reprezentarea lui -97:



Explicații complete:
https://www.cs.cornell.edu/~tomf/notes/cps104/twoscomp.html#whyworks

Reprezentarea în memorie

- char: ocupă 1 octet = **8 biți**, valori între -2^7=**-128** și 2^7-1=**127**

```
char ch = 'a';
```



'a' are codul ASCII 97 = $2^6 + 2^5 + 2^0$

- int: ocupă 4 octeți = **32 biți**, valori între -2^{31} și $2^{31}-1$

```
int i = 190;
```

Reprezentarea binară a lui 190 în memoria calculatorului:



$$190 = 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$$

Reprezentarea în memorie

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)

```
int i = 190;
```

Reprezentarea binară a lui 190 în memoria calculatorului:



$190 = 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$ (scrierea în baza 2 a lui 190)

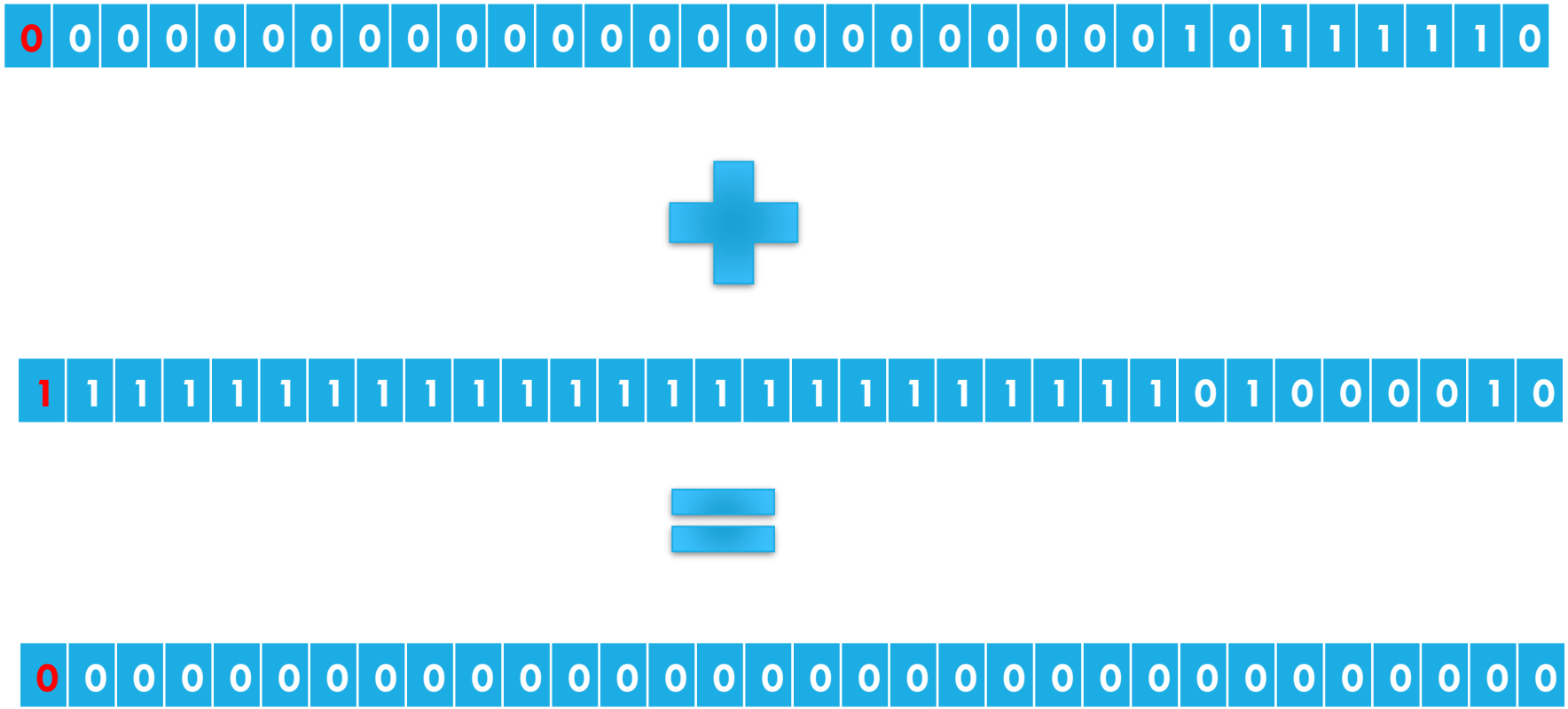
Cum se reprezintă -190 în memoria calculatorului?



Reprezentarea binară a lui -190 în memoria calculatorului

Reprezentarea în memorie

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)



Reprezentarea în memorie

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)

190



-190



0

Modificatori de tip

- **signed**
  - ✓ modificatorul implicit pentru toate tipurile de date
  - ✓ *bitul cel mai semnificativ* din reprezentarea valorii *este semnul*
- **unsigned**
  - ✓ restricționează valorile numerice memorate la valori pozitive
  - ✓ domeniul de valori este mai mare deoarece bitul de semn este liber și participă în reprezentarea valorilor
- **short**
  - ✓ reduce dimensiunea tipului de date întreg la jumătate
  - ✓ se aplică doar pe întregi
- **long**
  - ✓ permite memorarea valorilor care depășesc limita specifică tipului de date
  - ✓ se aplică doar pe int sau double: la int dimensiunea tipului de bază se dublează, la double se mărește dimensiunea cu cel putin doi octeți
- **long long**
  - ✓ in C99: pentru stocarea unor valori întregi de dimensiuni foarte mari

Tipuri de date + modificatori

Tip de date + modificator	Dimensiune în biți	Domeniu de valori
char	8	de la -128 la 127
unsigned char	8	de la 0 la 255
signed char	8	de la -128 la 127
int (signed int)	32	de la -2^{31} la $2^{31}-1$
unsigned int	32	de la 0 la $2^{32}-1$
short int (signed short int)	16	de la -2^{15} la $2^{15}-1$
unsigned short int	16	de la 0 la $2^{16}-1$
long int	64	de la -2^{63} la $2^{63}-1$
float	32	…
double	64	…

CUPRINSUL CURSULUI DE AZI

1. Tipuri de date fundamentale
2. Variabile și constante
3. Expresii și operatori

VARIABILE ȘI CONSTANTE

- stochează datele necesare programului
  - ✓ valorile stocate în memoria sistemului în mod transparent de către programator
  - ✓ referirea la aceste date se face prin numele lor simbolice (**identificatori)**

- **variabilele** stochează date care pot fi modificate în timpul execuției
- **constantele** păstrează aceeași valoare (cea cu care au fost inițializate) până la terminarea programului

VARIABILE

- se caracterizează prin:
  - ✓ un **nume** (identificator),
  - ✓ un **tip**,
  - ✓ o **valoare**,
  - ✓ **adresa de memorie** unde se află stocată valoarea variabilei,
  - ✓ **domeniu de vizibilitate.**
- oricărei variabile i se alocă un spațiu de memorie corespunzător tipului variabilei

Exemplu: **int notaExamen =10;**

int = tipul variabilei (de obicei se va stoca pe 32 de biți)

notaExamen = numele variabilei

10 = valoarea variabilei

&notaExamen = adresa din memorie unde se află stocată valoarea variabilei cu numele notaExamen

DOMENIUL DE VIZIBILITATE A VARIABILELOR

- domeniul de vizibilitate a unei variabile = porțiunea de cod la a cărei execuție variabila respectivă este accesibilă
- Variabile:
  - ✓ locale – vizibile local, numai în funcția sau blocul de instrucțiuni unde au fost declarate
  - ✓ globale – vizibile global, din orice zonă a codului
- parametrii formali ai unei funcții se comportă asemenea unor variabile locale.

VARIABILE LOCALE

- variabilele definite în corpul unei funcții sau al unui bloc de instrucțiuni sunt locale (vizibile) acelei funcții sau bloc.

```c
#include <stdio.h>

void f1()
{
    int x=10;
    printf("In functia f1 valoarea lui x este %d.\n", x);
}
void f2()
{
    int x=15;
    printf("In functia f2 valoarea lui x este %d.\n", x);
}
int main()
{
    int x=30;
    f1();
    f2();
    printf("In functia main valoarea lui x este %d.\n", x);

    return 0;
}
```

```
In functia f1 valoarea lui x este 10.
In functia f2 valoarea lui x este 15.
In functia main valoarea lui x este 30.
```

VARIABILE LOCALE

- variabile definite în corpul unei funcții sau al unui bloc de instrucțiuni sunt locale (vizibile) acelei funcții sau bloc.

```
 1  #include <stdio.h>
 2
 3  int main()
 4  {
 5      int i;
 6      for(i=0; i<10; i++)
 7      {
 8          int j = 2*i;
 9          printf("j = %d \n",j);
10      }
11      printf("j = %d \n", j);
12
13      return 0;
14  }
```

```
In function 'main':
11  error: 'j' undeclared (first use in this function)
11  error: (Each undeclared identifier is reported only once
```

Eroare la linia 11: variabila j nu a fost declarată.
Ea este vizibilă numai în blocul de instrucțiuni anterior.

VARIABILE LOCALE

- variabile definite în corpul unei funcții sau al unui bloc de instrucțiuni sunt locale (vizibile) acelei funcții sau bloc.

```c
#include <stdio.h>

int main()
{
    int i;
    for(i=0; i<10; i++)
    {
        int j = 2*i;
        printf("j = %d \n",j);
    }
    int j = i;
    printf("j = %d \n", j);

    return 0;
}
```

```
j = 0
j = 2
j = 4
j = 6
j = 8
j = 10
j = 12
j = 14
j = 16
j = 18
j = 10
```

VARIABILE GLOBALE

- se declară în afara oricărei funcții și sunt vizibile în întreg programul
- pot fi accesate de către orice zonă a codului
- orice expresie are acces la ele, indiferent de tipul blocului de cod în care se află expresia

VARIABILE GLOBALE

```c
#include <stdio.h>

int x = 10;
void f1(int x){
    x = x + 10;
    printf("In functia f1 valoarea lui x este %d.\n", x);}
void f2(){
    x = x + 20;
    printf("In functia f2 valoarea lui x este %d.\n", x);}
void f3(int x){
    x = x + 30;
    printf("In functia f3 valoarea lui x este %d.\n", x);}

int main()
{
    f1(x);
    x = x * 5;
    f2();
    f3(x);
    printf("In functia main valoarea lui x este %d.\n", x);
    return 0;
}
```

```
In functia f1 valoarea lui x este 20.
In functia f2 valoarea lui x este 70.
In functia f2 valoarea lui x este 100.
In functia main valoarea lui x este 70.
```

La apelul funcțiilor f1 și f3 se realizează o copie locală a lui x.
După ieșirea din f1, copia se distruge. Întrucât f1 nu întoarce nicio valoare, x rămâne cu valoarea de înainte de apelarea lui f1.

VARIABILE GLOBALE

```
1  #include <stdio.h>
2
3  int x = 10;
4  void f1(int x)
5  {
6      x = x + 10;
7      printf("In functia f1 valoarea lui x este %d.\n", x);
8  }
9  void f2(int x)
10 {
11     x = x + 20;
12     printf("In functia f2 valoarea lui x este %d.\n", x);
13 }
14 int main()
15 {
16     f1(x);
17     int x = 5;
18     f2(x);
19     printf("In functia main valoarea lui x este %d.\n", x);
20     return 0;
21 }
```

```
In functia f1 valoarea lui x este 20.
In functia f2 valoarea lui x este 25.
In functia main valoarea lui x este 5.
```

Variabila locala (linia 17) ia locul variabilei globale

CONSTANTE ÎNTREGI

- **zecimale** (baza 10, prima cifră nenulă): 1234
- **octale** (baza 8, prima cifră 0): 01234
- **hexazecimale** (baza 16, prefixul 0x sau 0X): 0xFA; 0XABBA

- Efectul sufixului adăugat unei constante întregi (în funcție de valoare):
  - ✓ U sau u: unsigned int sau unsigned long int: 52u, 400000U
  - ✓ L sau l: long int: 52L, 32000L
  - ✓ UL sau uL sau Ul sau ul: unsigned long int: 52uL, 400000Ul

CONSTANTE ÎNTREGI

```c
#include <stdio.h>

int main()
{
    int x;

    x = 123;
    printf("%d \n", x);

    x = 0123;
    printf("%d \n", x);

    x = 0xAA;
    printf("%d \n", x);
    return 0;
}
```

```
123
83
170
```

CONSTANTE ÎN VIRGULĂ MOBILĂ

- compuse din:
  - ✓ semn,
  - ✓ parte întreagă,
  - ✓ punctul zecimal,
  - ✓ parte fracționară,
  - ✓ marcajul pentru exponent (e sau E)
- partea întreagă sau fracționară pot lipsi (dar nu ambele)
- punctul zecimal sau marcajul exponențial pot lipsi (dar nu ambele)
- format aritmetic: 3.1415; .2; 7.;
- format exponențial: 31415E-4; 6.023E+23
- implicit constantele în virgulă mobilă sunt stocate ca double

CONSTANTE ÎN VIRGULĂ MOBILĂ

- partea întreagă sau fracționară pot lipsi (dar nu ambele)
- punctul zecimal sau marcajul exponențial pot lipsi (dar nu ambele)
- format aritmetic:

  3.1415; .2; 7.;

- format exponențial:

  31415E-4; 6.023E+23

```c
#include<stdio.h>

int main()
{
  float pi=3.1415, a=.2, b=7.;
  printf("\t%f %f %f", pi, a, b);

  float x = 31415E-4, y = 6E+3;
  printf("\n\t%.4f %.4f", x, y);

  return 0;
}
```

```
3.141500 0.200000 7.000000
3.1415 6000.0000
```

CONSTANTE CARACTER

- au ca valoarea codul ASCII al caracterelor pe care le reprezintă
- caractere imprimabile:
    - ✓ caractere grafice (coduri ASCII între 33 și 126)
    - ✓ spațiu (cod ASCII = 32)
- o constantă caracter corespunzătoare unui caracter imprimabil se reprezintă prin caracterul respectiv inclus între caractere apostrof:

    'a' (codul ASCII 97), 'A' (codul ASCII 65)
- cum se reprezintă caracterele apostrof și backslash?
    - ✓ apostrof = '\''; backslash = '\\';

IDENTIFICATORI

- fiecare constantă și variabilă trebuie să aibă un nume unic

Reguli:

- sunt permise doar **literele alfabetului**, **cifrele** şi _(underscore)
- identificatorul nu poate începe cu o cifră
  - ✓ nu putem declara variabile având numele: 2win, 01_nr, etc.
- literele mari sunt tratate diferit de literele mici
  - ✓ Maxim, maxim, maXim şi MaxiM desemnează variabile diferite
- numele nu poate fi cuvânt cheie al limbajului C
  - ✓ nu putem declara variabile având numele for, while, exit, etc.

CUPRINSUL CURSULUI DE AZI

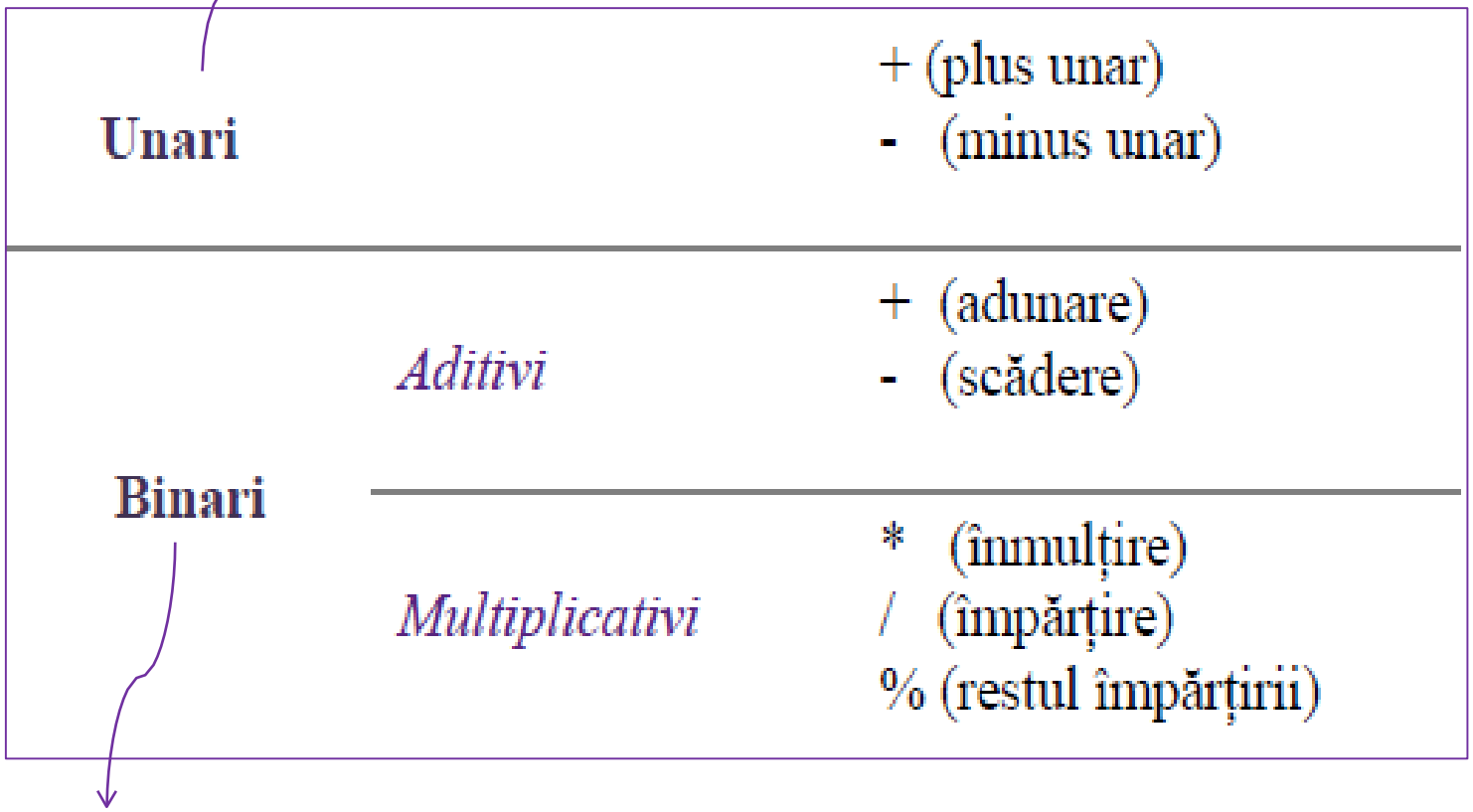
1. Tipuri de date fundamentale
2. Variabile și constante
3. Expresii și operatori

EXPRESII ȘI OPERATORI

- **expresii**
  - ✓ sunt formate din **operanzi** și **operatori**
  - ✓ arată modul de calcul al unor valori
  - ✓ cea mai simplă expresie este formată dintr-un operand.
- **operatori**
  - ✓ elemente fundamentale ale expresiilor
  - ✓ operatori aritmetici, relaționali, etc.
  - ✓ Limbajul C are foarte mulți operatori (46 în tabelul de la sfârșit)
- **operanzi**
  - ✓ variabile, constante
  - ✓ apel de funcție
  - ✓ expresii între paranteze
  - ✓ etc.

EXPRESII ARITMETICE ȘI OPERATORI ARITMETICI

Se aplică asupra unui singur operand



Necesită doi operanzi

EXPRESII ARITMETICE ȘI OPERATORI ARITMETICI

Exemple

```
int a, b, c = +3;      // operatorul unar +
b = -4;                // operatorul unar -
a = b - c + 1;         // operatorul binar - și +    a este -6
a = a * b / 2;         // operatorul binar * și /    a este 12
c = a % 5;             // operatorul binar %         c este 2
```

- operatorii aritmetici se pot aplica asupra operanzilor
  - ✓ de tip întreg (int, char) sau
  - ✓ de tip real (float sau double)

- se pot combina aceste tipuri de date în aceeași expresie
  - ✓ **excepție**: % doar între întregi

EXPRESII ARITMETICE ȘI OPERATORI ARITMETICI

Observații:

- Operatorul **/** semnifică:
  - ✓ împărțirea întreagă dacă ambii operanzi sunt întregi (int, char)
  - ✓ împărțirea cu virgulă dacă cel puțin unul dintre operanzi este de tip real (float, double)

```
int a = 5, b = 2;
float x = 5.0f;
a = a / b;      // a este 2
x = x / b;      // x este 2.5
x = 5 / b;      // x este 2.0
```

- Împărțirea la zero:
  - ✓ operatorii / și % **nu** pot avea operandul din dreapta 0

- Trunchierea la împărțirea întreagă:

```
c =   7 /  5;      // c este  1 (trunchiat de la 1.4)
c =  -7 /  5;      // c este -1 (trunchiat de la -1.4)
c =   9 /  5;      // c este  1 (trunchiat de la 1.8)
c =   9 / -5;      // c este -1 (trunchiat de la -1.8)
```

- **În C/C++ nu există un operator pentru ridicarea la putere!**

EVALUAREA EXPRESIILOR

- principii fundamentale pentru evaluarea oricăror expresii prin intermediul expresiilor aritmetice:
- **precedența** și **asociativitatea** operatorilor
  - ✓ dacă într-o expresie apar mai mulți operatori, atunci evaluarea expresiei respectă **ordinea de precedență** a operatorilor
  - ✓ dacă într-o expresie apar mai mulți operatori de aceeași prioritate, atunci se aplică **regula de asociativitate** a operatorilor: de la stânga la dreapta, *cu excepția operatorilor unari, condiționali și de atribuire, care se asociază de la dreapta la stânga*.

OPERATORI

1. Operatori aritmetici
2. Operatorul de atribuire
3. Operatori de incrementare și decrementare
4. Operatori de egalitate, logici și relaționali
5. Operatori pe biți
6. Alți operatori:
   - de acces la elemente unui tablou,
   - de apel de funcție,
   - de adresa,
   - de referențiere,
   - sizeof,
   - de conversie explicită,
   - condițional,
   - virgulă.

2. OPERATORI DE ATRIBUIRE

- operatorul de atribuire simplă **=**
  - ✓ efect: evaluarea expresiei din dreapta operatorului și asignarea acestei valori la variabila din stânga operatorului

```
a = 10;                 // a ia valoarea 10
b = a;                  // b ia valoarea 10
c = a + (b-7) * 3;      // c ia valoarea 19
```

- valoarea unei atribuiri **var = expresie** este valoarea lui var după asignare
- expresia de atribuire poate să apară ca operand într-o altă expresie unde se așteaptă o valoare de tipul var

```
a = 3;
b = 5 - (c = a);      // c ia valoarea 3 care
                      // se scade din 5 și astfel b devine 2
```

  - ✓ *expresia devine greu de înțeles și poate introduce erori greu de depistat*

OPERATORI DE ATRIBUIRE

- atribuirea formalizată: expr1 = expr2
  - ✓ expr1 este *lvalue* (valoare stânga)
    - ✓ trebuie să permită stocarea valorii lui expr2 în memorie
      - ➢ corect: v[i+1]=10
      - ➢ *incorect: 10 = v[i+1]*

- dacă tipul lui expr1 și expr2 nu este același, atunci se aplică regula conversiei implicite
  - ✓ valoarea lui expr2 este convertită la tipul lui expr1 în momentul asignării

```
int a;
float x;
a = 12.34f;              // a ia valoarea 12
x = 123;                 // x ia valoarea 123.0
```

CONVERSII

- Dacă toți operanzii care intervin într-o expresie au același tip: tipul expresiei coincide cu tipul operanzilor
- **Conversii implicite:** la evaluarea unei expresii în care operanzii nu au același tip: operandul care are un domeniu de valori mai restrâns este convertit la tipul operandului care are mulțimea valorilor mai amplă.
- **Conversii explicite**: operatorul (tip)

```
(5 + 2)/2 = 3
(5 + 2)/2 = 3.50
(5 + 6.00)/2 = 5.50
```

```c
#include<stdio.h>

int main()
{
    int a = 5, b = 2; float c = 6;
    printf("(%d + %d)/2 = %d \n", a, b, (a + b)/2);
    printf("(%d + %d)/2 = %.2f \n", a, b, (float)(a + b)/2);
    printf("(%d + %d)/2 = %.2f \n", a, b, (a + b)/2.);

    printf("(%d + %.2f)/2 = %.2f \n", a, c, (a + c)/2);

    return 0;
}
```

CONVERSII

```c
#include<stdio.h>
#include<limits.h>

int main()
{
   printf("%d\n", INT_MAX);
   int i = 2000000000;
   printf("2 * %d = %d\n", i, 2*i);
   printf("2 * %d = %ld\n", i, 2*i);
   printf("2 * %d = %ld\n", i, (long)2*i);
   long d = i + i;
   printf("d = %ld\n", d);

   return 0;
}
```

valoarea obtinuta in urma depasirii tipului int este atribuita lui d

```
    9 |     printf("2 * %d = %ld\n", i, 2*i);//warning!
      |                       ~~^         ~~~
      |                         |         |
      |                         long int  int
      |                       %d
2147483647
2 * 2000000000 = -294967296
2 * 2000000000 = 4000000000
2 * 2000000000 = 4000000000
d = -294967296
```

OPERATORI DE ATRIBUIRE

- regula de asociativitate
  - ✓ atribuirile se pot înlănțui

```
a = b = c = 0;
```

- operatori de atribuire compuși (operator =)
  - ✓ exemple : **+=, -=, *=, /=, %=,** șamd. (combinat cu operatori pe biți)
  - ✓ permit calcularea noii valori a variabilei folosind valoarea veche a acesteia

```
a += 1;          // a se incrementează cu 1: a = a + 1;
b -= 3;          // asemănător cu b = b - 3;
c *= 4;          // asemănător cu c = c * 4;
```

  - ✓ dar nu este întotdeauna echivalent cu varianta descompusă
    - ➢ contează ordinea de precedență și efectele secundare

```
a *= b + c;      // nu este echivalent cu a = a * b + c;
                 // este echivalent cu  a = a * (b + c);
```

3. OPERATORI DE INCREMENTARE ȘI DECREMENTARE

- operatorii **++** și **--**
  - ✓ incrementarea/decrementarea unei variabile cu 1

- forma prefixă (++i sau --i)
  - ✓ preincrementare/predecrementare

- forma postfixă (i++ sau i--)
  - ✓ postincrementare/postdecrementare

```
i++;
```

Exemplu echivalent:

```
i = i + 1;
i += 1;
```

- efect secundar: modificarea valorii operandului:
  - valoarea returnată:
    - ✓ preincrementarea (++a) returnează valoarea a+1
    - ✓ postincrementarea (a++) returnează valoarea a

OPERATORI DE INCREMENTARE ȘI DECREMENTARE

```
int a = 5, b = 2, c;
c = a - ++b;            // ⇔ b = b+1;  c = a-b;
                        // valorile a: 5, b: 3, c: 2
c = ++a + b--;          // ⇔ a = a+1;   c = a + b;  b = b-1;
                        // valorile a: 6, b: 2, c: 9
```

- operatorii de **pre**incrementare și **pre**decrementare au aceeași prioritate ca și operatorii unari **+** și **-**
- operatorii de **pos**tincrementare și **pos**tdecrementare au prioritate crescută în raport cu operatorii unari **+** și **-**

EXPRESII LOGICE

- se evaluează la valori de tip *adevărat* sau *fals*
- sunt construite cu ajutorul a trei categorii de operatori
  - ✓ operatori **relaționali**
  - ✓ operatori de **egalitate**
  - ✓ operatori **logici**
- limbajul C tratează valorile *adevărat* și *fals* ca valori întregi:
  - ✓ 0 înseamnă fals
  - ✓ orice altă valoare nenulă se interpretează ca adevărat

OPERATORI RELAȚIONALI

- operatorii <, >, <=, >=
- rezultatul este o valoare logică, adică valoarea 0 (fals) sau 1 (adevărat)
- sunt mai puțin prioritari decât operatorii aritmetici

```
5   < 10          // rezultat: 1
10  <  5          // rezultat: 0
3   > 2.5         // rezultat: 1
                  // se pot combina tipurile întreg și real
a + b <= c - 1    // este de fapt (a + b) <= (c - 1)
                  // respectând ordinea de precedență
```

```
a < b < c  // echivalent cu (a < b) < c
           // datorita asociativitatii stanga
```

4. OPERATORI DE EGALITATE

- testează egalitatea dintre două valori

  == este operatorul ”egal cu”

  != este operatorul ”diferit de”

- generează o valoare logică: 0 (fals) sau 1 (adevărat)
- în ordinea de precedență a operatorilor sunt mai puțin prioritari decât operatorii relaționali

```
a == 2          // returnează 1 dacă a este 2,
                // 0 în caz contrar
a != b          // returnează 1 dacă a nu este egal cu b,
                // 0 dacă a și b au valori identice
a < b == b < c  // este echivalent cu (a < b) == (b < c)
                // returnează 1 doar dacă expresiile au
                // aceeași valoare:
                // ambele sunt adevărate sau ambele false
```

5. OPERATORI LOGICI

- limbajul C furnizează 3 operatori logici:

  ! – non/ not (negarea logică), operatorul unar

```
!expr      // 1 dacă expr are valoarea logică 0 (fals)
           // 0 dacă expr are valoarea logică nenulă (adevărat)
```

  && - ȘI logic (conjuncția logică), operator binar

```
expr1 && expr2   // este 1 dacă expr1 și expr2 sunt nenule
```

  || - SAU logic (disjuncția logică), operator binar

```
expr1 || expr2   // este 1 dacă expr1 sau expr2 este nenulă
```

- generează o valoare logică: 0 (fals) sau 1 (adevărat)

OPERATORI LOGICI

- Evaluarea unei expresii:
  - ✓ dacă se poate deduce rezultatul global din evaluarea expresiei din stânga, atunci expresia din dreapta nu se mai evaluează

```
(a != 0) && (a % 4 == 0)
```

- operatorul ! (negare logică) are prioritate egală cu cea a operatorilor aritmetici unari (+ și -)

- operatorii && și || sunt mai puțin prioritari decât operatorii relaționali și cei de egalitate

6. OPERATORI PE BIȚI

- două categorii:

✓ **operatori logici pe biți:**

& ȘI pe biți, operator binar

| SAU pe biți, operator binar

^ SAU EXCLUSIV pe biți, operator binar

~ complement față de 1, operator unar

✓ **operatori de deplasare pe biți:**

<< deplasare stânga pe biți, operator binar

>> deplasare dreapta pe biți, operator binar

- operanzi de tip întreg (nu merg pe float, double)
- ordinea de precedență - în cadrul acestei categorii:

Prioritate crescută	~	(complement față de unu)
	<<	(deplasare stânga)
	>>	(deplasare dreapta)
	&	(și pe biți)
	^	(sau exclusiv pe biți)
Prioritate scăzută	|	(sau pe biți)

OPERATORI PE BIȚI

- & seamănă cu &&
- | seamănă cu ||
- au un rol similar, dar la nivelul fiecărei perechi de biți de pe poziții identice
- ~ este echivalentul operației ! dar aplicat la nivel de biți

Expresie	Reprezentare pe 4 biți				Observație
a = 10	1	0	1	0	
b = 7	0	1	1	1	
a & b	0	0	1	0	1 dacă ambi biți sunt 1, 0 în rest
a | b	1	1	1	1	1 dacă cel puțin unul din cei doi biți este 1, 0 în rest
a ^ b	1	1	0	1	1 dacă doar unul din cei doi biți este 1, 0 în rest
~ a	0	1	0	1	1 unde bitul a fost 0 și 0 unde bitul a fost 1
~ b	1	0	0	0	1 unde bitul a fost 0 și 0 unde bitul a fost 1

OPERATORI DE DEPLASARE PE BIȚI

- condiții:
  - ✓ **operanzi întregi**
  - ✓ al doilea operand cu valoare mai mică (nu negativ) decât numărul de biți pe care este reprezentat operandul din stânga
- **deplasarea spre stânga** ⇔ **înmulțire** cu 2 la puterea deplasamentului
- **deplasarea spre dreapta** ⇔ **împărțire** cu 2 la puterea deplasamentului

Expresie	Reprezentare binară	Observație
a = 12	0000 0000 0000 1100	
b = 3600	0000 1110 0001 0000	
a << 1	0000 0000 0001 10**00**	Valoarea rezultată este 24 = 12 * 2^1
a << 2	0000 0000 0011 0**000**	Valoarea rezultată este 48 = 12 * 2^2
a << 5	0000 0001 10**00 0000**	Valoarea rezultată este 384 = 12 * 2^5
a >> 1	**0**000 0000 0000 0110	Valoarea rezultată este 6 = 12 / 2^1
a >> 2	**00**00 0000 0000 0011	Valoarea rezultată este 3 = 12 / 2^2
b >> 4	**0000** 0000 1110 0001	Valoarea rezultată este 225 = 3600 / 2^4

7. ALȚI OPERATORI

- operatorul de acces la elementele tabloului [ ]

```
int a[100];
a[5] = 10;
```

- operatorul de apel de funcție (): b = f(a);
- operatorul adresă & și operatorul de dereferențiere *
  - ✓ strâns legat de pointeri (cursurile următoare)

```
int a, *p;          // p este un pointer la int
p = &a;             // p este pointer la a
*p = 3;             // valoarea lui a devine 3
```

- operatorul sizeof

sizeof (a) = numărul de octeți ocupați în memorie de a

- operatorul de conversie explicită: (tip)

```
int a = 1, b = 2;
float media;

media = ( a + (float)b ) / 2;    // media devine 1.5
media = ( a + b ) / 2;           // media devine 1.0  - incorect!
```

ALȚI OPERATORI

- operatorul condițional ? :
  - ✓ operator ternar
  - ✓ similar cu instrucțiunea **if**
  - ✓ **`expresie1? expresie2 : expresie3`**
  - ✓ dacă `expresie1` e adevarată, execută `expresie2`, altfel execută `expresie3`

```
int a=3, b=5, max;
max = a > b ? a : b;
```

```
a % 2 ? printf("numar impar") : printf("numar par");
```

- operatorul virgulă
  - ✓ evaluarea secvențială a expresiilor (de la stânga la dreapta)
  - ✓ valoarea ultimei expresii din înlănțuire este valoarea expresiei compuse
  - ✓ cel mai puțin prioritar din lista de precedență

```
int i, n, s;
printf("n=");scanf("%d", &n);
for(i=1,s=0; i<=n; s=s+i, i=i+1);
```

Operator	Description	Associativity
( ) [ ] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

CURSUL 2. CUPRINS:

1. Tipuri de date fundamentale
2. Variabile și constante
3. Expresii și operatori

CURSUL 3. CUPRINS:

1. Pointeri
2. Tablouri. Șiruri de caractere.
3. Structuri, uniuni, câmpuri de biți, enumerări.